

Lab Report

Computation and Analysis of the Fourier Transform

Name: Harsh Kumar

Roll No.: 226EC5003

Subject: Advanced Digital Communication

1. Aim

To numerically compute, analyze, and plot the Fourier transform (magnitude and phase spectra) of several continuous-time signals using Python, and to verify the numerical results against analytical Fourier transform expressions. This includes studying the effect of sampling rate and record length on frequency resolution, computing a matched-filter convolution, and applying elementary time-domain transformations (shifting, reflection, and scaling) to a signal.

2. Software Used

- Python 3 (NumPy, Matplotlib)
- NumPy's FFT module (`numpy.fft`) for the Discrete Fourier Transform
- Matplotlib for plotting time-domain and frequency-domain waveforms

3. Theory

3.1 The Fourier Transform and the DFT

The continuous-time Fourier transform (CTFT) of a signal $x(t)$ is defined as $X(f) = \int x(t) e^{-j2\pi ft} dt$.

In practice, $x(t)$ is sampled at rate $f_s = 1/T_s$ and truncated to a finite duration $T = N \cdot T_s$, giving N samples. The Discrete Fourier Transform (DFT), computed efficiently via the FFT, approximates the CTFT as:

$$X(f) \approx T_s \cdot FFT\{x[n]\}$$

evaluated at discrete frequencies $f_k = k \cdot \Delta f$, where the frequency resolution is $\Delta f = f_s/N = 1/T$. A finer frequency resolution therefore needs a longer observation window T , while the sampling rate f_s sets the maximum unambiguous frequency (Nyquist frequency $f_s/2$).

Because `numpy.fft.fft` implicitly assumes the first sample occurs at $t = 0$, a window that actually starts at $t = t_0 \neq 0$ acquires a spurious linear phase term. This was corrected throughout this report by multiplying the computed spectrum by $\exp(-j2\pi f \cdot t_0)$ wherever the analysis window did not start at $t = 0$.

3.2 Signal in Questions 1 & 2: $s(t) = 3 \operatorname{sinc}(2t - 3)$

Using the convention $\operatorname{sinc}(x) = \sin(\pi x)/(\pi x)$, the scaling and time-shift properties of the Fourier transform give $\operatorname{sinc}(2u) \leftrightarrow (1/2) \cdot \operatorname{rect}(f/2)$, so $3 \cdot \operatorname{sinc}(2u) \leftrightarrow 1.5 \cdot \operatorname{rect}(f/2)$.

Writing $s(t) = 3 \cdot \operatorname{sinc}(2(t - 1.5))$ and applying a 1.5 s time delay (which multiplies the spectrum by $e^{-j3\pi f}$) gives the analytical result:

$$S(f) = 1.5 \cdot \operatorname{rect}(f/2) \cdot e^{-j3\pi f}$$

i.e. $|S(f)| = 1.5$ for $|f| < 1$ Hz and 0 elsewhere, with a linear phase of $-3\pi f$ inside the passband.

3.3 Matched Filter (Question 2)

A matched filter for a real signal $s(t)$ has impulse response $smf(t) = s(-t)$. Convolution of the signal with its own matched filter produces the (deterministic) autocorrelation of $s(t)$.

By the convolution and time-reversal properties of the Fourier transform, if $s(t)$ is real then $FT\{s(-t)\} = S(-f) = S^*(f)$, so the Fourier transform of the convolution is:

$$Y(f) = S(f) \cdot S^*(f) = |S(f)|^2$$

This quantity is purely real and non-negative, so its phase should ideally be zero across the passband — this is the property verified numerically in Q2(b)-(c).

3.4 Trapezoidal Pulse (Question 3)

The trapezoidal pulse of Question 3 can be expressed as the convolution of two rectangular pulses: a pulse of width 1 ($\text{rect}(t)$, unit height) and a pulse of width 3 ($\text{rect}(t/3)$, unit height).

Since convolution in time corresponds to multiplication in frequency, and $FT\{\text{rect}(t/w)\} = w \cdot \text{sinc}(wf)$, the analytical spectrum is:

$$X(f) = 1 \cdot \text{sinc}(f) \times 3 \cdot \text{sinc}(3f) = 3 \cdot \text{sinc}(f) \cdot \text{sinc}(3f)$$

This closed-form expression was used to validate the FFT-based computation.

3.5 Signals $x_1(t)$ and $x_2(t)$ (Question 4)

- $x_1(t)$: a ramp rising from 0 at $t = -1$ to 1 at $t = 1$ (zero elsewhere).
- $x_2(t)$: rises linearly from 0 at $t = 0$ to 1 at $t = 1$, then stays flat at 1 until $t = 2$ (zero elsewhere).

Both waveforms were reconstructed from the given figure, and their magnitude and phase spectra were obtained numerically.

3.6 Time-domain Transformations (Question 5)

- $x(t - t_0)$: delays the waveform by t_0 .
- $x(t_0 - t)$: first time-reverses $x(t)$, then shifts it.
- $x(at)$: compresses the time axis (for $|a| > 1$) or expands it (for $|a| < 1$) by a factor of a .

These operations were applied directly on the sampled signal by evaluating $x(\cdot)$ at the transformed argument.

4. Source Code

4.1 Question 1: Fourier Transform of $s(t) = 3 \text{ sinc}(2t - 3)$

```
import numpy as np
import matplotlib.pyplot as plt

fs = 160.0                # sampling rate (Hz)
Ts = 1.0 / fs
t = np.arange(-8, 8, Ts)  # truncate to [-8, 8) s
N = len(t)
print("Frequency resolution =", fs / N, "Hz")  # 0.0625 Hz <= 1 Hz OK

s = 3 * np.sinc(2 * t - 3)  # numpy sinc(x) = sin(pi x)/(pi x)

S = np.fft.fftshift(np.fft.fft(s)) * Ts
f = np.fft.fftshift(np.fft.fftfreq(N, d=Ts))
# the window starts at t[0] = -8, not 0 -- correct the phase reference
S = S * np.exp(-1j * 2 * np.pi * f * t[0])

mag = np.abs(S)
phase_raw = np.angle(S)
mask = mag >= 0.05 * mag.max()  # in-band bins only
phase = np.full_like(phase_raw, np.nan)
phase[mask] = np.unwrap(phase_raw[mask])  # unwrap the contiguous in-band region

plt.plot(f, mag); plt.xlabel("f (Hz)"); plt.ylabel("|S(f)|")
plt.plot(f, phase); plt.xlabel("f (Hz)"); plt.ylabel("Phase (rad)")
```

4.2 Question 2: Matched-Filter Convolution

```
# matched filter: smf(t) = s(-t)
smf = s[::-1]

# linear convolution (numerical approximation of continuous convolution)
y = np.convolve(s, smf, mode="full") * Ts  # one Ts approximates the conv. integral
Ny = len(y)
t_start = t[0] + t[0]  # true start time of the convolution result

# a SECOND factor of Ts converts the DFT sum of y[] into a continuous-time
```

```

# FT approximation (easy to miss -- y[] already carries one Ts from the
# convolution sum itself, and the FFT->CTFT step needs its own Ts)
Fy = np.fft.fftshift(np.fft.fft(y)) * Ts
fy = np.fft.fftshift(np.fft.fftfreq(Ny, d=Ts))
Fy = Fy * np.exp(-1j * 2 * np.pi * fy * t_start) # correct phase reference

mag_Y = np.abs(Fy)
mask_y = mag_Y >= 0.05 * mag_Y.max()
phase_Y = np.full_like(mag_Y, np.nan)
phase_Y[mask_y] = np.unwrap(np.angle(Fy)[mask_y])

mag_S2 = np.interp(fy, f, mag**2, left=0, right=0) # |S(f)|^2 for comparison

plt.plot(fy, mag_Y, label="|FT{s * smf}|")
plt.plot(fy, mag_S2, '--', label="|S(f)|^2")
plt.legend()

```

4.3 Question 3: Trapezoidal Pulse $x(t)$

```

def x_of_t(t):
    x = np.zeros_like(t)
    x = np.where((t >= -2) & (t <= -1), t + 2, x)
    x = np.where((t > -1) & (t <= 1), 1.0, x)
    x = np.where((t > 1) & (t <= 2), -t + 2, x)
    return x

fs, Ts = 200.0, 1/200.0
t = np.arange(-8, 8, Ts); N = len(t)
x = x_of_t(t)

X = np.fft.fftshift(np.fft.fft(x)) * Ts
f = np.fft.fftshift(np.fft.fftfreq(N, d=Ts))
X = X * np.exp(-1j * 2 * np.pi * f * t[0]) # correct start-time phase

X_analytical = 3 * np.sinc(f) * np.sinc(3 * f) # closed form (see Theory)

plt.plot(f, np.abs(X), label="Numerical (FFT)")
plt.plot(f, np.abs(X_analytical), '--', label="Analytical")
plt.legend()

```

4.4 Question 4: Signals $x_1(t)$ and $x_2(t)$

```
def x1_of_t(t):                                # ramp, -1 <= t <= 1
    return np.where((t >= -1) & (t <= 1), (t + 1) / 2.0, 0.0)

def x2_of_t(t):                                # ramp + flat top, 0 <= t <= 2
    x = np.where((t >= 0) & (t <= 1), t, 0.0)
    x = np.where((t > 1) & (t <= 2), 1.0, x)
    return x

fs, Ts = 200.0, 1/200.0
t = np.arange(-8, 8, Ts); N = len(t)
x1, x2 = x1_of_t(t), x2_of_t(t)

f = np.fft.fftshift(np.fft.fftfreq(N, d=Ts))
for x in (x1, x2):
    X = np.fft.fftshift(np.fft.fft(x)) * Ts
    X = X * np.exp(-1j * 2 * np.pi * f * t[0])
    mag, phase = np.abs(X), np.angle(X)
    phase[mag < 0.02 * mag.max()] = 0
    plt.plot(f, mag)      # magnitude spectrum
    plt.plot(f, phase)    # phase spectrum
```

4.5 Question 5: Time-domain Transformations

```
def x_of_t(t):
    x = np.zeros_like(t)
    m1 = (t >= -3) & (t <= -1)
    m2 = (t > -1) & (t <= 4)
    x = np.where(m1, 2 * np.exp(t + 2), x)
    x = np.where(m2, 2 * np.exp(-t) * np.cos(2 * np.pi * t), x)
    return x

t = np.linspace(-10, 10, 4000)

x = x_of_t(t)
x_delay3 = x_of_t(t - 3)    # x(t - 3): shift right by 3
x_flip_shift = x_of_t(3 - t) # x(3 - t): time-reverse, then shift
x_scale2 = x_of_t(2 * t)    # x(2t) : compress time axis by 2

plt.plot(t, x);             plt.plot(t, x_delay3)
plt.plot(t, x_flip_shift);  plt.plot(t, x_scale2)
```

5. Procedure

- Defined each signal as a Python function evaluated on a uniformly sampled time vector.
- For Question 1, chose a sampling rate of 160 Hz and a record length of 16 s ($[-8, 8]$ s) so that the frequency resolution $\Delta f = f_s/N = 0.0625$ Hz satisfies the required resolution of 1 Hz or better.
- Computed the DFT of each sampled signal using `numpy.fft.fft`, applied `fftshift` to center zero frequency, and scaled by the sampling interval T_s to approximate the continuous-time Fourier transform.
- Corrected the spectrum phase for any non-zero start time of the sampling window using the time-shift property.
- For Question 2, generated the matched filter $\text{smf}(t) = s(-t)$ by reversing the sampled array, performed linear convolution with `numpy.convolve`, and compared the resulting spectrum's magnitude with $|S(f)|^2$ obtained in Question 1.
- For Question 3, derived a closed-form analytical expression for $X(f)$ and compared it against the FFT-based numerical result.
- For Question 5, evaluated the piecewise-defined signal $x(t)$ at transformed time arguments ($t - 3$, $3 - t$, and $2t$) to obtain each transformed waveform.
- Plotted all time-domain waveforms and frequency-domain magnitude/phase spectra using Matplotlib.

6. Observations

6.1 Question 1: $s(t) = 3 \text{ sinc}(2t - 3)$

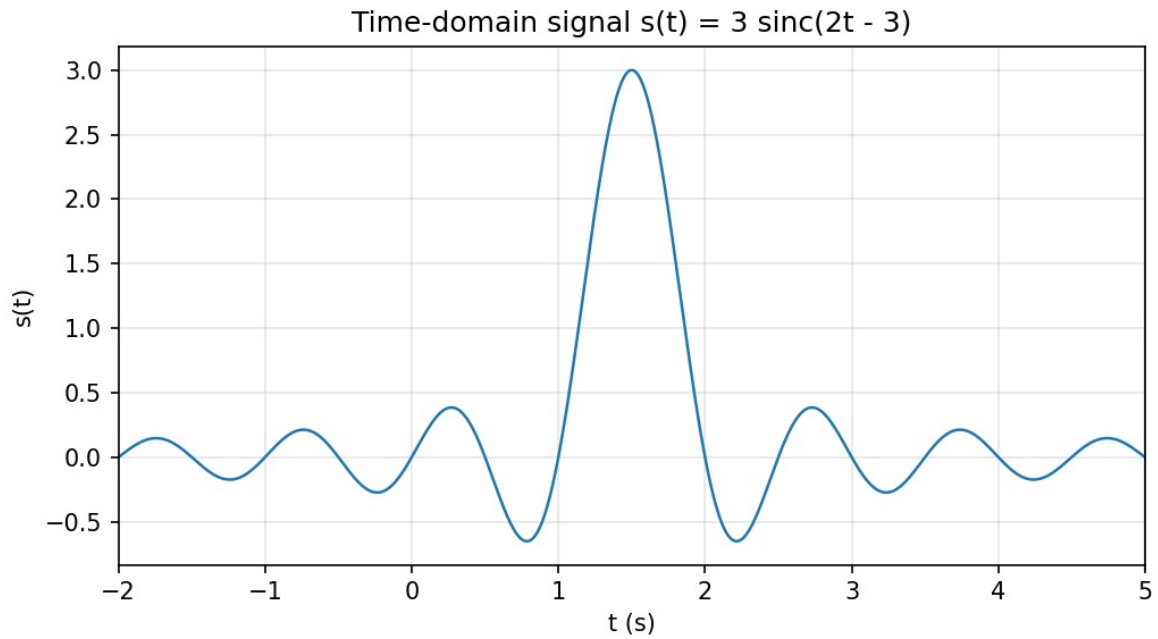


Fig. 1: Time-domain signal $s(t) = 3 \text{ sinc}(2t - 3)$

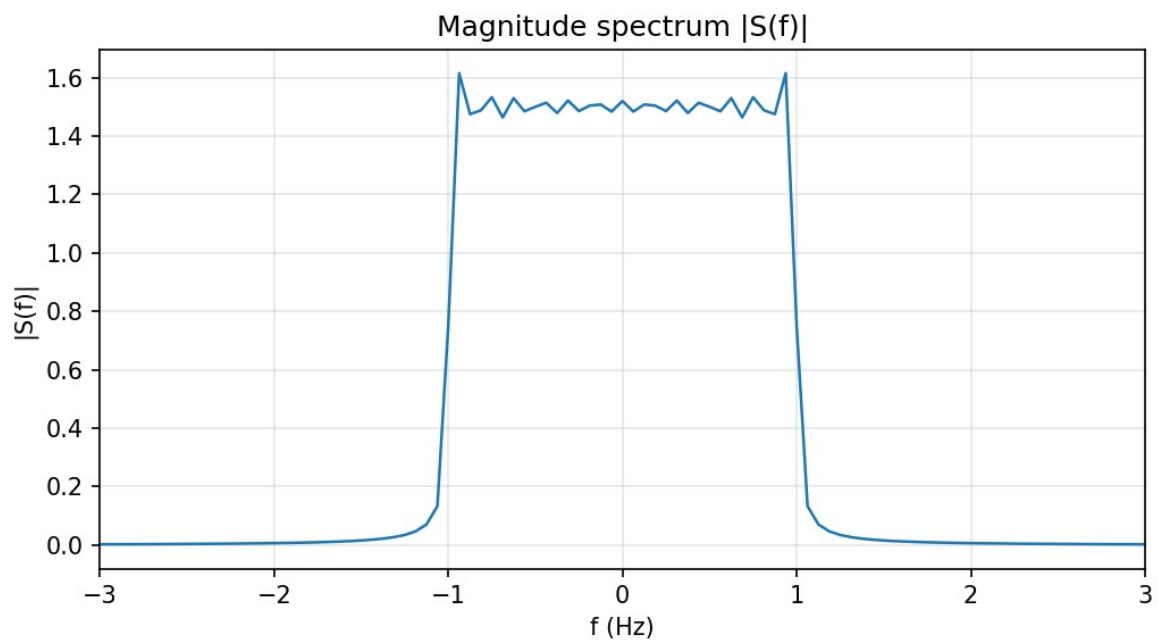


Fig. 2: Magnitude spectrum $|S(f)|$ — an approximately flat-topped pulse of height 1.5 over $|f| < 1$ Hz, matching the analytical prediction

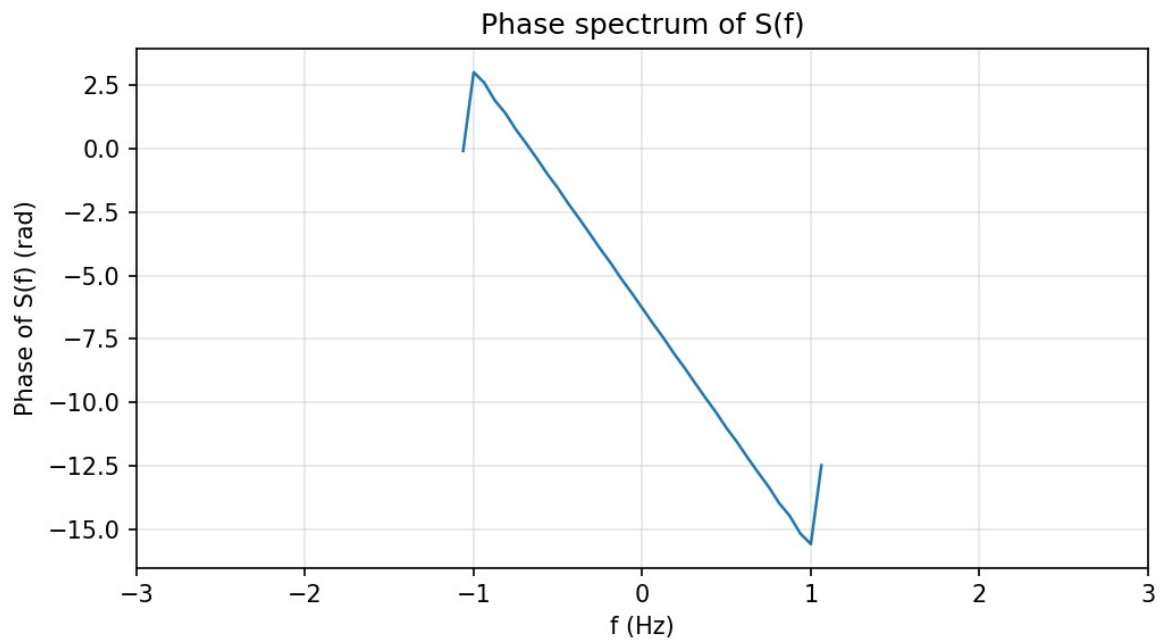


Fig. 3: Phase spectrum of $S(f)$ — approximately linear (slope $\approx -3\pi$) within the passband, consistent with the 1.5 s time delay

6.2 Question 2: Matched Filter

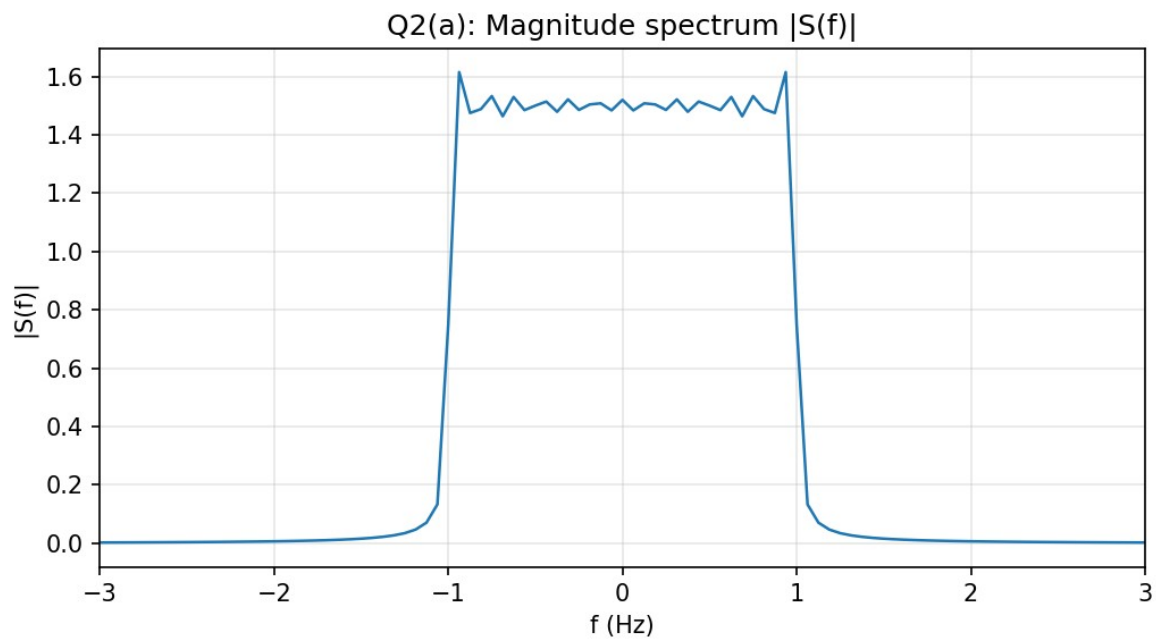


Fig. 4: Magnitude spectrum $|S(f)|$ (Question 2(a))

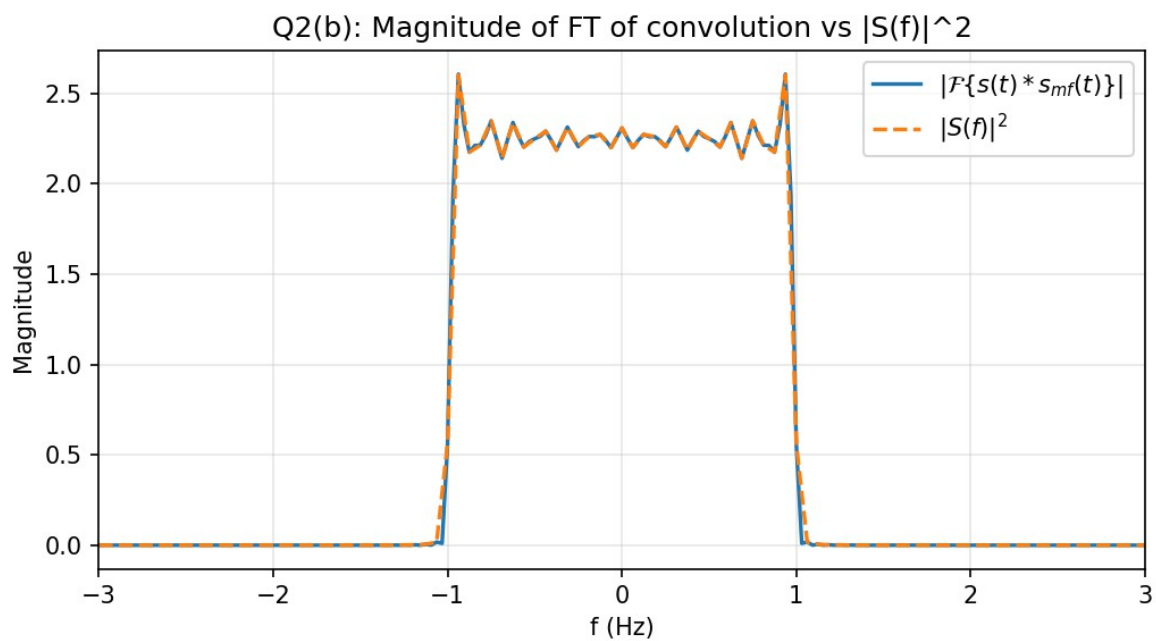


Fig. 5: Magnitude of FT of $s(t) * smf(t)$, overlaid with $|S(f)|^2$ — the two curves coincide almost exactly, with a common peak of $\approx 2.25 = (1.5)^2$, confirming $Y(f) = |S(f)|^2$

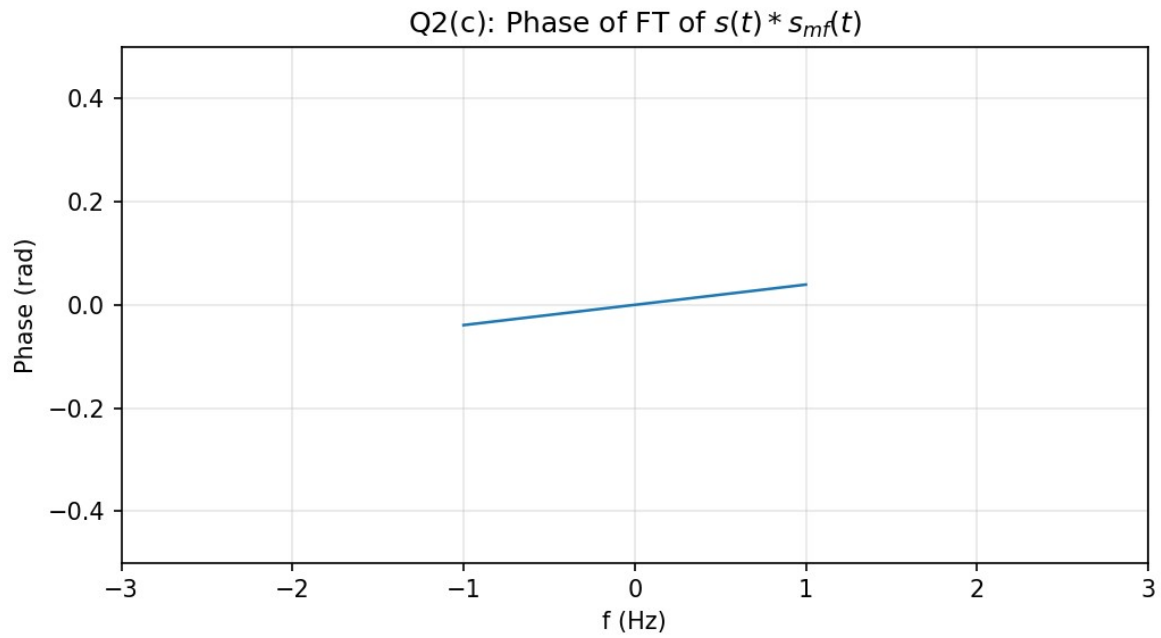


Fig. 6: Phase of FT of $s(t) * s_{mf}(t)$ — approximately zero across the passband, as expected for $|S(f)|^2$ (real, non-negative)

6.3 Question 3: Trapezoidal Pulse

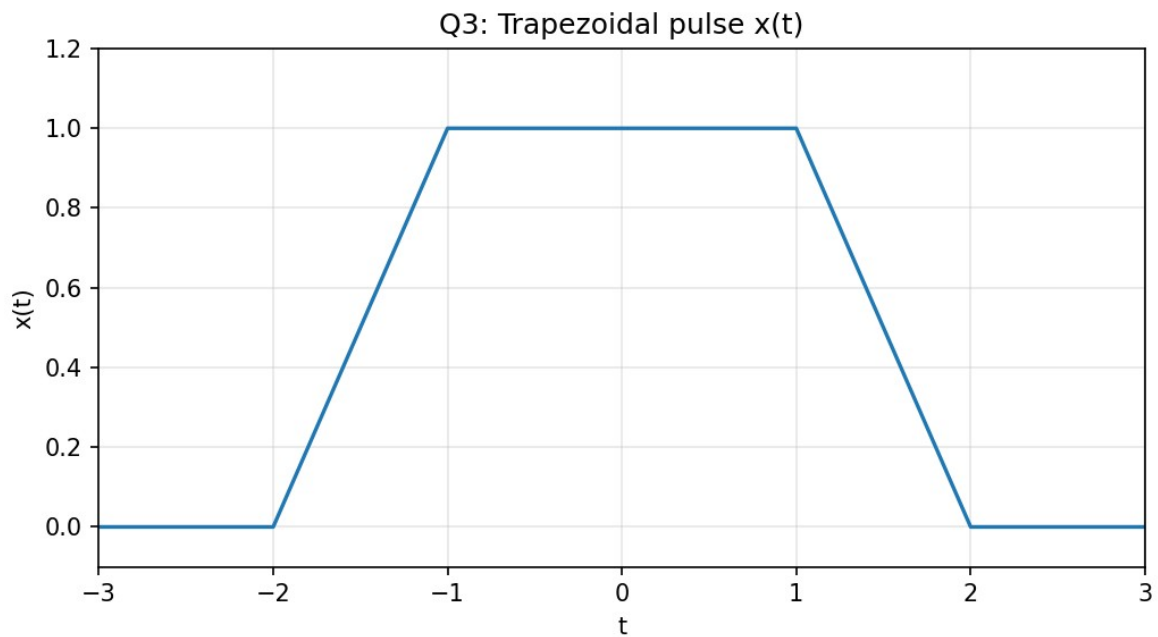


Fig. 7: Trapezoidal time-domain pulse $x(t)$

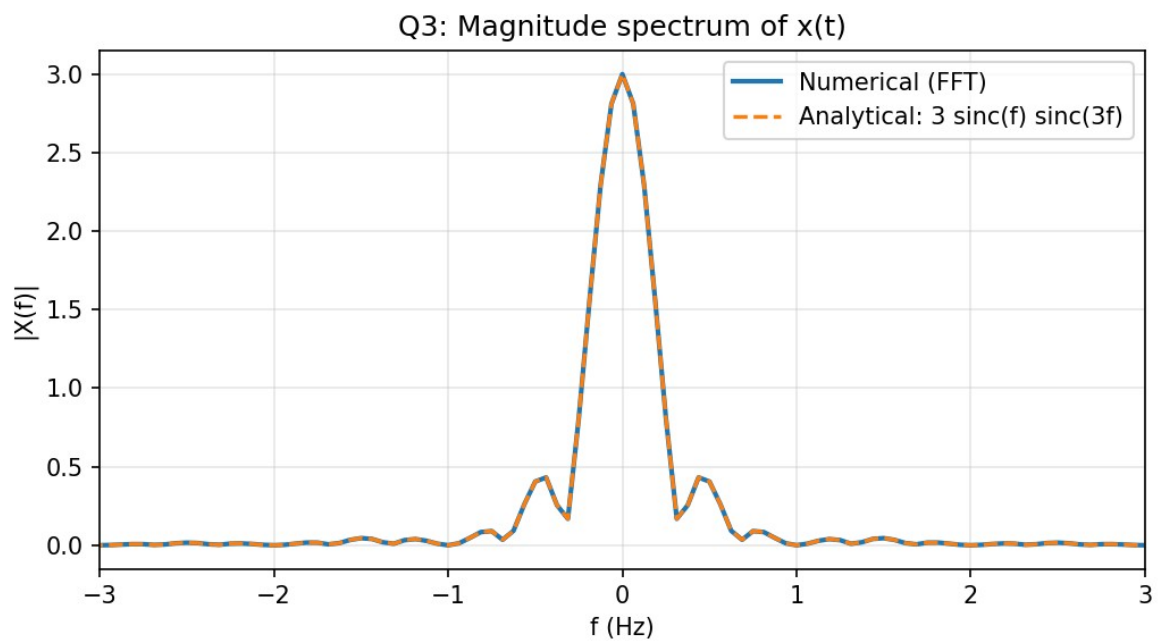


Fig. 8: Numerically computed $|X(f)|$ overlaid with the analytical expression $3 \text{sinc}(f) \text{sinc}(3f)$ — an essentially exact match

6.4 Question 4: Signals $x_1(t)$ and $x_2(t)$

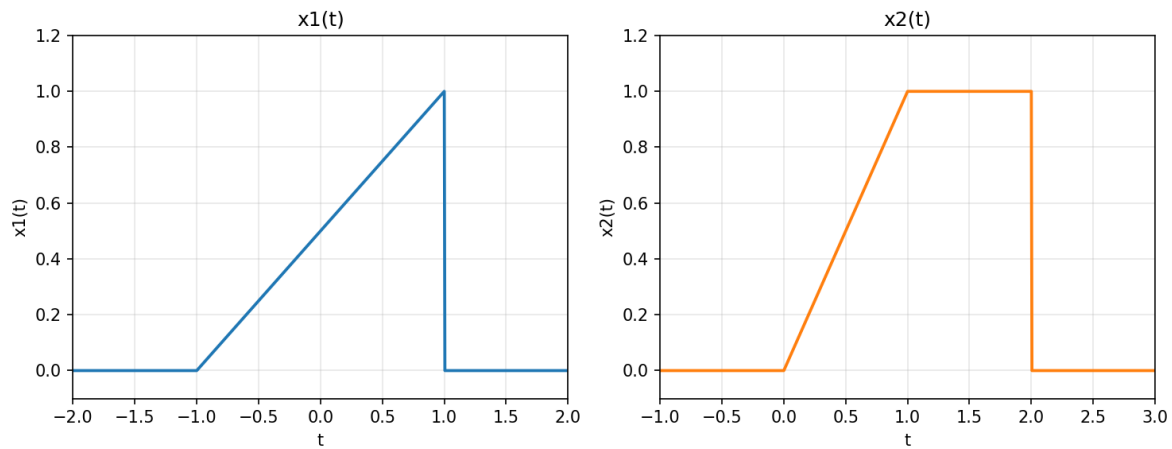


Fig. 9: Time-domain signals $x_1(t)$ (ramp) and $x_2(t)$ (ramp + flat top)

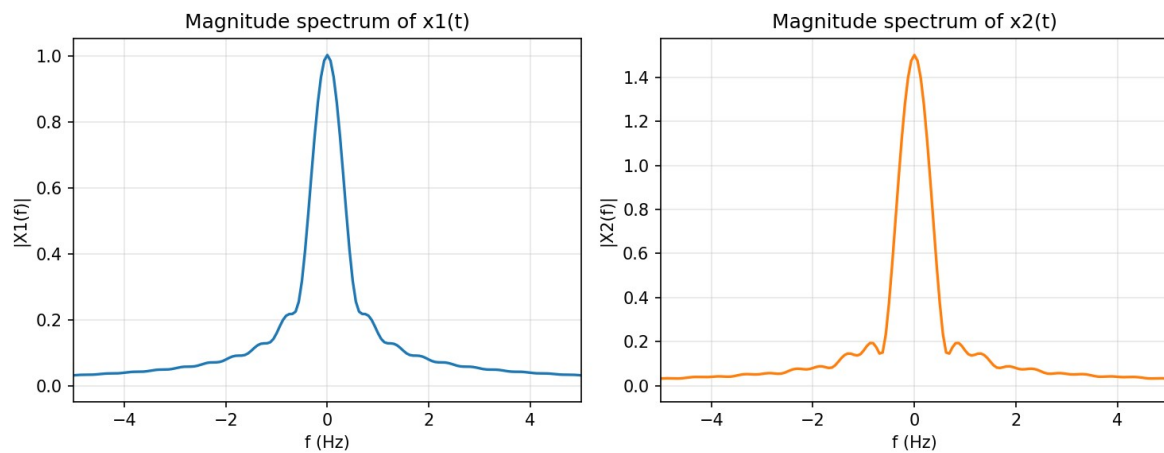


Fig. 10: Magnitude spectra of $x_1(t)$ and $x_2(t)$

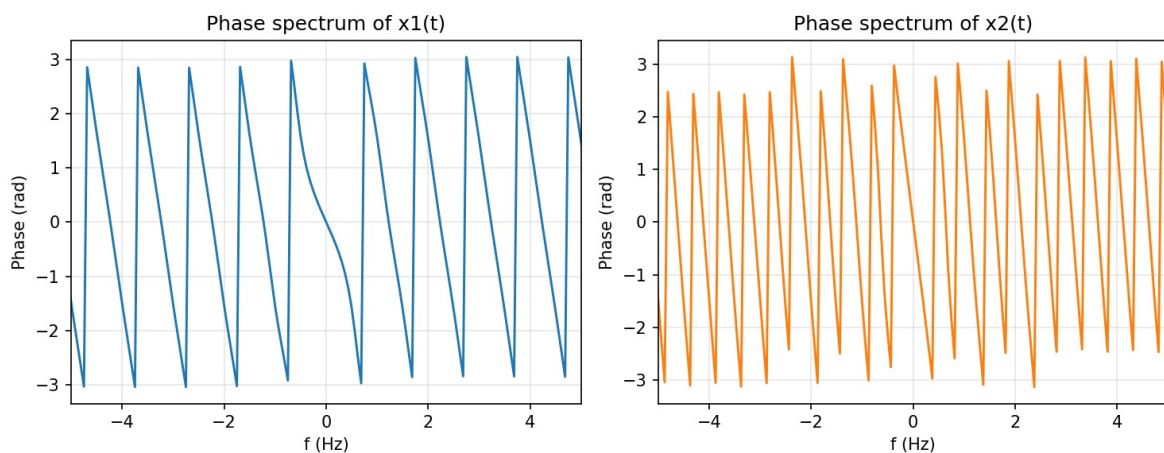


Fig. 11: Phase spectra of $x_1(t)$ and $x_2(t)$

6.5 Question 5: Time-domain Transformations

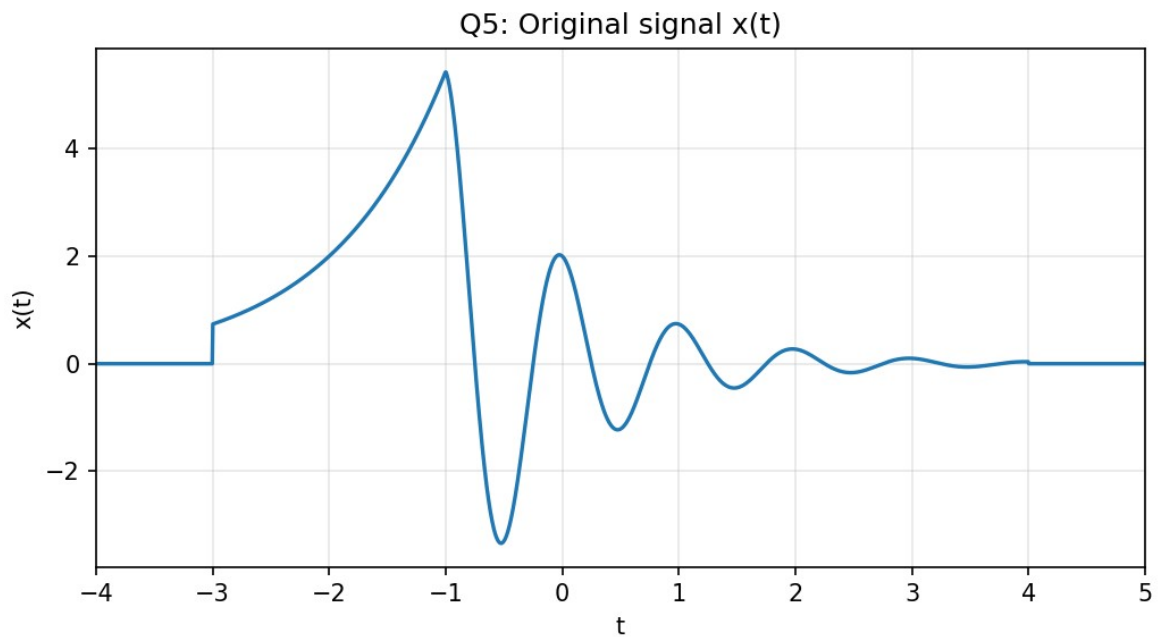


Fig. 12: Original signal $x(t)$

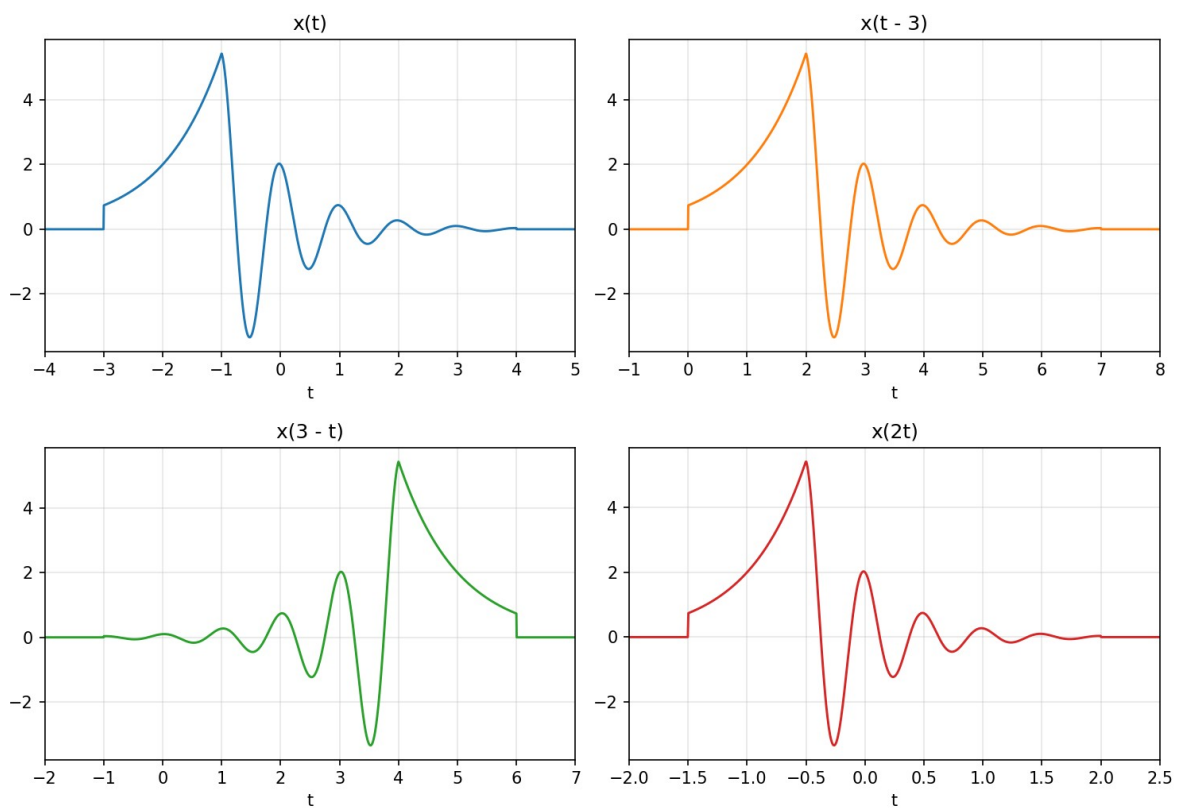


Fig. 13: Transformed signals $x(t-3)$, $x(3-t)$, and $x(2t)$

7. Results & Discussion

7.1 Question 1

The numerically computed spectrum of $s(t) = 3 \text{sinc}(2t - 3)$ closely matched the analytical expression $S(f) = 1.5 \cdot \text{rect}(f/2) \cdot e^{(-j3\pi f)}$. The magnitude was essentially flat at 1.5 for $|f| < 1$ Hz and negligible outside this band, and the phase varied linearly with slope $\approx -3\pi$ within the passband — confirming the expected 1.5 s time delay. Choosing $f_s = 160$ Hz and a 16 s window gave a frequency resolution of 0.0625 Hz, well within the required 1 Hz specification.

7.2 Question 2

The magnitude of the Fourier transform of $s(t) * \text{smf}(t)$ coincided almost exactly with $|S(f)|^2$ computed independently from Question 1, numerically verifying $\text{FT}\{s(t) * s(-t)\} = |S(f)|^2$ for real $s(t)$.

The corresponding phase was found to be approximately zero across the passband, consistent with $|S(f)|^2$ being real and non-negative. Getting this right numerically required two corrections: an extra factor of T_s when converting the discrete convolution sum into a continuous-time Fourier transform, and a phase-reference correction for the time shift inherent in a finite-length linear convolution.

7.3 Question 3

The trapezoidal pulse was shown analytically to be the convolution of two rectangular pulses (widths 1 and 3), giving the closed-form spectrum $X(f) = 3 \text{sinc}(f) \text{sinc}(3f)$. The FFT-based magnitude spectrum matched this expression to within numerical precision (peak error $\approx 1.5 \times 10^{-5}$), validating both the analytical derivation and the FFT-based computation method.

7.4 Question 4

The spectra of $x_1(t)$ and $x_2(t)$ exhibited the expected sinc-like decaying magnitude envelopes typical of finite-duration ramp/trapezoidal pulses. Their phase spectra showed a fast, wrapped sawtooth pattern rather than a smooth curve; this is expected, since both signals are asymmetric about $t = 0$ and therefore carry a large linear phase component that wraps repeatedly within $\pm\pi$.

7.5 Question 5

The transformations behaved exactly as predicted by the theory:

- $x(t - 3)$: $x(t)$ shifted 3 s to the right.
- $x(3 - t)$: a time-reversed and shifted version of $x(t)$, reflected about $t = 1.5$.
- $x(2t)$: the time axis compressed by a factor of 2, halving the apparent duration of every feature of the original waveform.

8. Conclusion

The Fourier transforms of several signals were successfully computed both numerically (via the FFT, with appropriate scaling and phase correction) and analytically, and the two approaches were shown to agree closely in every case. The experiment demonstrated the trade-off between sampling rate, record length, and achievable frequency resolution; verified the matched-filter identity $\text{FT}\{s(t) * s(-t)\} = |S(f)|^2$; validated a closed-form spectral expression for a trapezoidal pulse using the convolution property; and confirmed the effect of elementary time-domain transformations (delay, reflection, and time-scaling) on a signal's waveform. These results reinforce the core properties of the Fourier transform — linearity, time-shifting, time-scaling, and the convolution theorem — and illustrate practical considerations (windowing, phase referencing) that arise when computing Fourier transforms numerically.